

AIOps Module 3 Assignment

Kirthan S, DA24B009

1. Single-stage and multi-stage Docker images

Both images built successfully and served `/healthz` and `/predict`. The multi-stage image produced the same responses as the naive image.

Image	Size
Naive single-stage image	2.1 GB
Multi-stage image	841 MB

The reduction is approximately $((2100 - 841)/2100) \times 100 = 60.0\%$. The naive image uses the full `python:3.12` base and copies the complete project into the final image. The multi-stage runtime image uses `python:3.12-slim` and keeps only the API code, trained `spam_model.joblib`, and installed runtime packages. The dataset, generator, training script, and the builder stage's temporary dependency wheels are not retained in the final image.

2. Docker Compose and Redis caching

Redis caching was added to `/predict`. The API uses the exact input text as part of its cache key. On a cache miss, it runs the model and stores the label with a one-hour TTL. On a hit, it returns the stored label without running the model again. The API and official `redis:7-alpine` image are defined as separate services in `docker-compose.yml`. Compose gives them a shared single-host network, so the API reaches Redis at the service hostname `cache`.

Request	Measured time
Cache miss	0.006944 seconds
Repeated cache hit	0.004029 seconds

The repeated request was about 42% faster. Compose solves the problem of running and connecting related containers on one machine. Kubernetes on the other hand, schedules and manages containers across a cluster, including replica maintenance and rolling updates.

3. Indexed Job for signup-data validation

Eight deterministic CSV shards containing malformed email addresses or missing required fields were generated. The Indexed Job has `completions: 8` and `parallelism: 4`. With the required capacity of two nodes with two CPUs each, four validator pods with a one-CPU request and limit use all four available CPUs. The shards therefore run in two waves of four. We use `restartPolicy: Never` therefore its CPU request equals its CPU limit.

The Job passes the completion index to each container through the Downward API. The validator maps index 0 through 7 to the corresponding shard. It also receives `spec.nodeName` through the Downward API and includes that node name in its log output. The saved `kubectl get pods -o wide` evidence shows four pods running concurrently, and the completed Job reported 8/8 successful completions.

Shard	0	1	2	3	4	5	6	7
Invalid rows	2	3	1	4	2	5	3	2

These results were collected with `kubectl logs`, which reads container logs through the Kubernetes API. A shared volume was not used because Minikube's default local-path storage is node-local, i.e., pods scheduled on different nodes might not see the same filesystem. With three two-CPU nodes, parallelism of 6 could be used. Six one-CPU validators fit within six CPUs, while parallelism 8 would require eight CPUs.

4. Deployment self-healing and rolling update

The spam API Deployment uses two replicas, CPU and memory requests and limits, a `/healthz` readiness probe, and a ClusterIP Service. The pod with a name ending in `jh5ld` was manually deleted and Kubernetes replaced it with a new pod ending in `h2csz`, restoring two ready pods. The Deployment manages a ReplicaSet with a desired count of two. When the count dropped to one, the ReplicaSet created a replacement. The readiness probe kept the replacement out of Service traffic until it returned HTTP 200.

For the rolling update, the health response was changed from version `v1` to `v2`, built `spam-api:v2`, and updated the Deployment image. Rollout status completed successfully, and rollout history recorded revision 2. The update uses `maxUnavailable: 0` and a readiness probe, so ready `v1` pods remain available until `v2` pods are ready. The Service returned version `v2` after the rollout.

A Job would not be appropriate for this API because it finishes after its pods complete and does not keep two serving replicas available. The Indexed Job is appropriate for the finite shard-validation task because each pod validates one shard and exits. The Deployment is appropriate for the long-running API because it maintains replicas, replaces failed pods, and performs gradual version updates.